



Decentralized Application Code Review And Security Analysis Report

Customer: DreamX

Date: 27/03/2026



We express our gratitude to the DreamX team for the collaborative engagement that enabled the execution of this dApp Security Assessment.

HODLBonds is a DeFi protocol where users deposit token pairs into vaults, and the protocol's trader swaps between them.

Document

Name	dApp Code Review and Security Analysis Report for DreamX
Audited By	Bogdan Bodisteanu
Approved By	Ece Orsel
Website	https://hodlbonds.io/
Changelog	26/03/2026 - Preliminary Report, 27/03/2026 - Final Report.
Platform	dApp
Language	TypeScript, Docker
Tags	dApp Audit
Methodology	https://docs.hacken.io/methodologies/dapp-audit-methodology

Review Scope

Repository	https://github.com/DreamX-Development/hodlbonds-blockchain-executor
Commit	35c049a (main branch)
Repository	https://github.com/DreamX-Development/hodlbonds-api
Commit	e8b188c (main branch)

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

9	9	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	1
High	3
Medium	1
Low	1

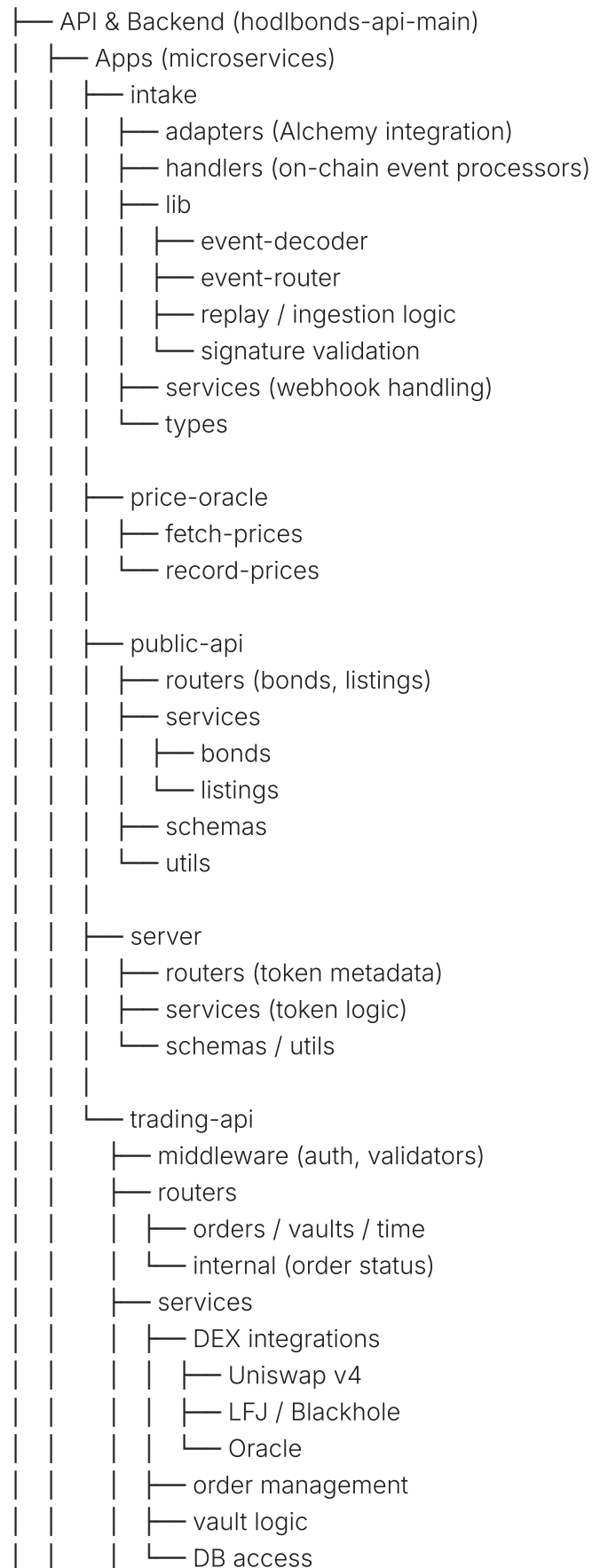
Vulnerability	Severity	Status
F-2026-15388 - Unauthenticated Administrative Debug Endpoints	Critical	Fixed
F-2026-15390 - Authentication Bypass via Replay Attack (Broken HMAC Middleware)	High	Fixed
F-2026-15401 - Plaintext Storage of Master Seed Phrase in Filesystem	High	Fixed
F-2026-15404 - Race Condition in Order Processing Leads to Inconsistent State and Potential Data Loss	High	Fixed
F-2026-15399 - Authentication Bypass on Internal Endpoints in Non-Production Environments	Medium	Fixed
F-2026-15393 - Observable Timing Discrepancy in Webhook Signature Validation	Low	Fixed
F-2026-15387 - Potential SQL Injection via Unparameterized sql.raw() Usage	Info	Fixed
F-2026-15395 - Missing Blockchain Transaction Receipt Status Validation	Info	Fixed
F-2026-15402 - Use of Cryptographically Weak Hash Algorithm (SHA-1)	Info	Fixed

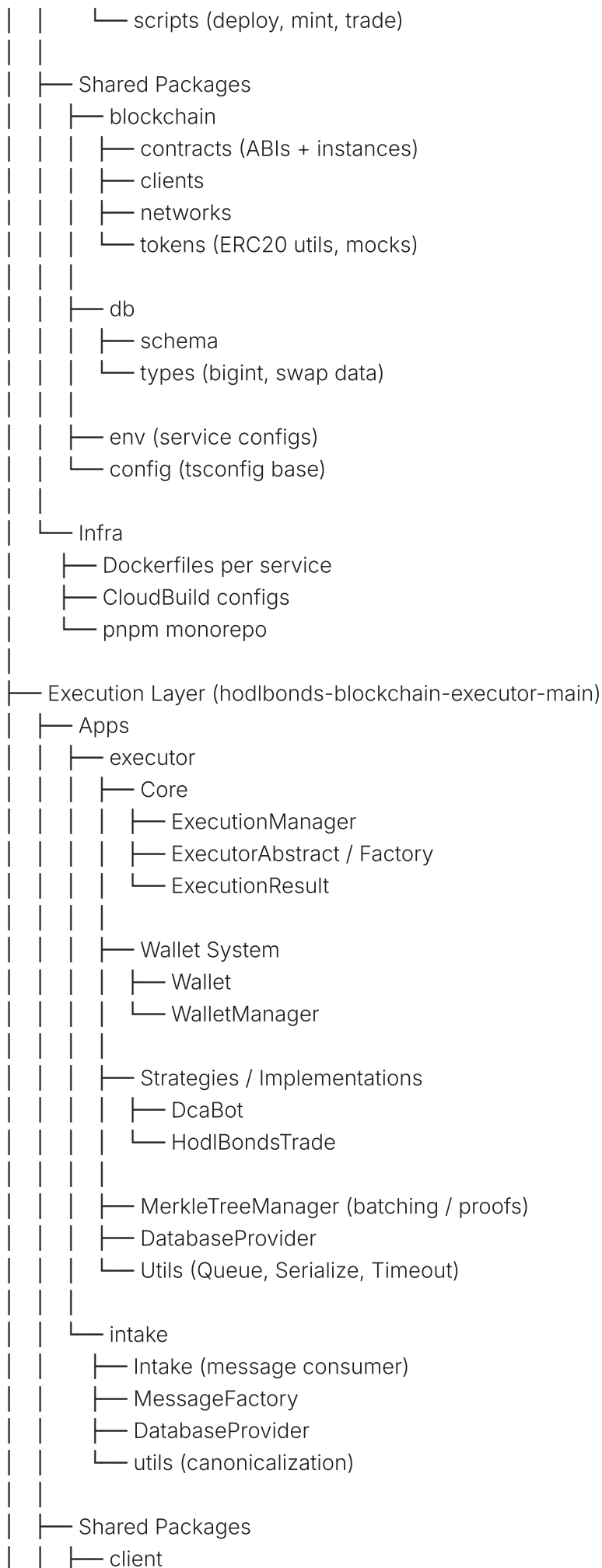
Table of Contents

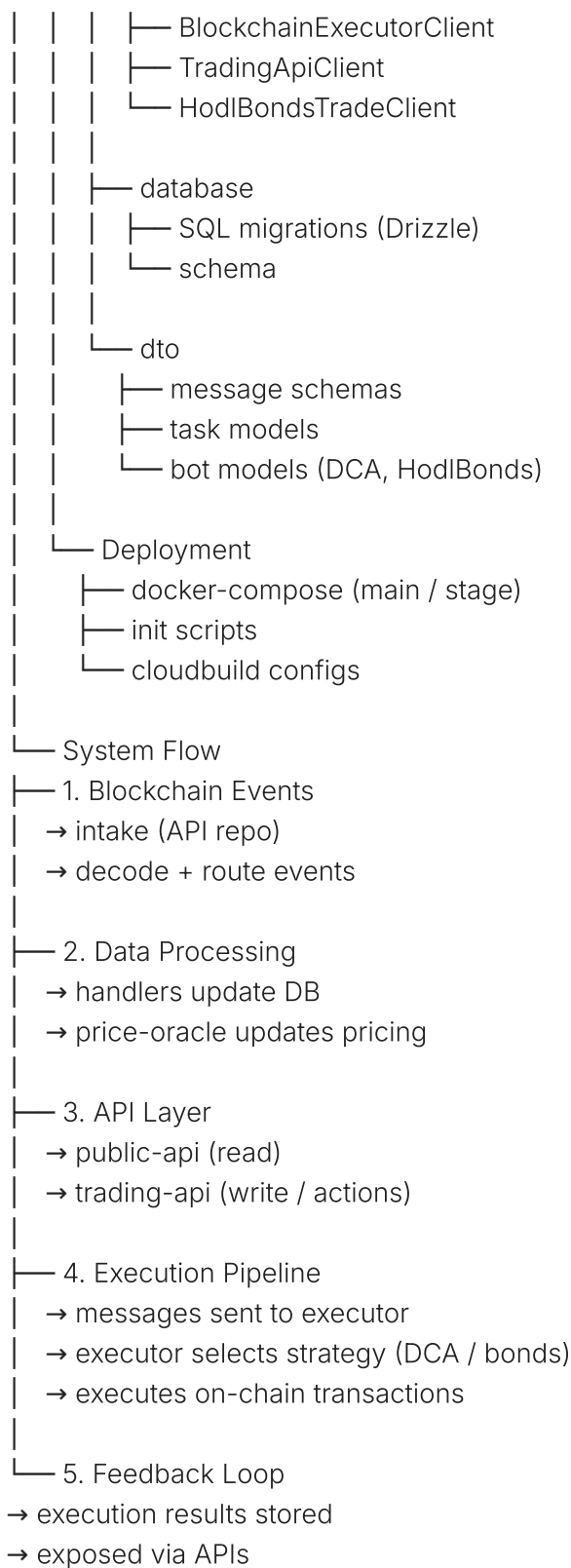
System Overview	5
Findings	8
Vulnerability Details	8
F-2026-15388 - Unauthenticated Administrative Debug Endpoints - Critical	8
F-2026-15390 - Authentication Bypass Via Replay Attack (Broken HMAC Middleware) - High	
F-2026-15404 - Race Condition In Order Processing Leads To Inconsistent State And Potential Data Loss - High	13
F-2026-15401 - Plaintext Storage Of Master Seed Phrase In Filesystem - High	15
F-2026-15399 - Authentication Bypass On Internal Endpoints In Non-Production Environments - Medium	18
F-2026-15393 - Observable Timing Discrepancy In Webhook Signature Validation - Low	21
F-2026-15387 - Potential SQL Injection Via Unparameterized Sql.Raw() Usage - Info	23
F-2026-15395 - Missing Blockchain Transaction Receipt Status Validation - Info	25
F-2026-15402 - Use Of Cryptographically Weak Hash Algorithm (SHA-1) - Info	27
Disclaimers	29
Hacken Disclaimer	29
Technical Disclaimer	29
Appendix 1. Severity Definitions	30
Appendix 2. Scope	31
Appendix 3. Additional Valuables	32
Frameworks And Methodologies	32

System Overview

HodlBonds System







Findings

Vulnerability Details

[F-2026-15388](#) - Unauthenticated Administrative Debug Endpoints - Critical

Description:

It has been found that multiple critical administrative endpoints in the intake application (apps/intake/src/index.ts) are exposed to the public without any authentication or authorization controls.

To understand this issue, it is helpful to clarify what "Administrative Endpoints" and "Authentication" mean. Administrative endpoints are special, hidden web addresses built into an application specifically for developers or system administrators. They act as a control panel to manage the system, fix stuck data, or manually force the system to process specific blockchain transactions. "Missing Authentication" means that the system operates like a bank vault with an open door and no security guard—it does not ask for a password, a secret token, or any proof of identity before executing these highly sensitive commands. Anyone who knows or guesses the web address can issue commands to the server.

Evidence

File: DreamX-Development___hodlbonds-api-main/apps/intake/src/index.ts

Code:

```
// ===== Admin/Debug Endpoints =====

// Get event processing stats
app.get("/admin/events/stats", async (c) => {
  try {
    const stats = await getEventStats()
    return c.json(stats)
  } catch (error) {
    console.error("Failed to get event stats:", error)
    return c.json({ error: error instanceof Error ? error.message : "Unknown error" }, 500)
  }
})

// Replay all failed events
app.post("/admin/events/replay", async (c) => {
  try {
    const limitParam = c.req.query("limit")
    const limit = limitParam ? Number(limitParam) : undefined

    console.log(`Admin: Replaying failed events${limit ? ` (limit: ${limit})` : ``} : ""`)
    await replayFailedEvents(limit)

    const stats = await getEventStats()
    return c.json({ message: "Replay complete", stats })
  } catch (error) {

```



```

console.error("Failed to replay events:", error)
return c.json({ error: error instanceof Error ? error.message : "Unknown error" }, 500)
}
})

// Replay a specific event
app.post("/admin/events/replay/:chainId/:txHash/:logIndex", async (c) => {
  try {
    const chainId = Number(c.req.param("chainId"))
    const txHash = c.req.param("txHash")
    const logIndex = Number(c.req.param("logIndex"))

    console.log(`Admin: Replaying event chainId=${chainId} txHash=${txHash} logIndex=${logIndex}`)
    await replayEvent(chainId, txHash, logIndex)

    return c.json({ message: "Event replayed successfully" })
  } catch (error) {
    console.error("Failed to replay event:", error)
    return c.json({ error: error instanceof Error ? error.message : "Unknown error" }, 500)
  }
})

// Ingest a transaction from the blockchain
app.post("/admin/events/ingest/:chainId/:txHash", async (c) => {
  try {
    const chainId = Number(c.req.param("chainId"))
    const txHash = c.req.param("txHash") as `0x${string}`

    if (!SUPPORTED_CHAINS.includes(chainId as any)) {
      return c.json({ error: `Unsupported chain: ${chainId}` }, 400)
    }

    console.log(`Admin: Ingesting transaction ${txHash} on chain ${chainId}`)
    const result = await ingestTransaction(chainId, txHash)

    return c.json({
      message: "Transaction ingested successfully",
      ...result,
    })
  } catch (error) {
    console.error("Failed to ingest transaction:", error)
    return c.json({ error: error instanceof Error ? error.message : "Unknown error" }, 500)
  }
})

```

In the provided code, there are four specific routes under the `/admin/*` path:

Looking at the code for these routes, there is no security middleware applied. For example, when a request hits `app.post("/admin/events/replay", async (c) => {`, the application immediately reads the limit parameter and calls the backend function `await replayFailedEvents(limit)`. There is no logic to verify *who* is making the request. In contrast, the `/webhooks/alchemy/:chainId` endpoint correctly checks an X-Alchemy-Signature header, but the admin routes are left completely unprotected.

Furthermore, the application is configured with a fully permissive Cross-Origin Resource Sharing (CORS) policy (`app.use("/*", cors({ origin: "*", ... })))`. This means any website on the internet can instruct a victim's web browser to send requests to these endpoints without the browser blocking it.

Assets:

- apps/trading-api (single app only) [https://github.com/hknio/DreamX-Development___hodlbonds-api]

Status:**Fixed****Classification****Vulnerability Type(s):**

- Denial of Service
- Gain Information
- Gain Privilege

CVSS 4.0:

9.3 (AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:H/VA:H/SC:N/SI:N/SA:N)

Weakness (CWE):

- [CWE-306 | Missing Authentication for Critical Function](#)

Severity:**Critical****Recommendations****Remediation:**

Restrict access to all /admin/* endpoints by implementing a robust authentication mechanism. Additionally, strict CORS policies should be enforced so that only trusted domains can interact with the API via a web browser.

Implement Authentication Middleware: Use a secure, randomly generated API Key passed via HTTP headers (e.g., Authorization: Bearer <ADMIN_API_KEY>).

Hono provides a built-in bearerAuth middleware for this exact purpose. Apply to Admin Group: Group all admin routes and apply the authentication middleware explicitly to them. Restrict CORS:

Change the global CORS configuration from origin: "*" to a strict list of allowed origins (e.g., your administrative frontend domains).

[F-2026-15390](#) - Authentication Bypass via Replay Attack (Broken HMAC Middleware) - High

Description:

It has been found an Authentication Bypass via Replay Attack in the HMAC authentication middleware of the trading-api (apps/trading-api/src/middleware/auth.ts).

To understand this issue, it is important to explain what a "Replay Attack" and "HMAC Authentication" are. In secure APIs, instead of sending a password with every request, the application signs the request with a secret key (HMAC) and includes a timestamp. This acts like a stamped, single-use ticket. A "Replay Attack" occurs when a malicious actor intercepts this valid ticket (the network request) and simply sends it to the server again. To prevent this, the server must do two things: check if the timestamp is recent (e.g., within the last 30 seconds), and remember that it has already seen this exact ticket so it cannot be used twice.

Evidence

File: DreamX-Development___hodlbonds-api-main/apps/trading-api/src/middleware/auth.ts

Code:

```
const seenTimestamps = new LRUCache<string, boolean>({
  max: 10_000,
  updateAgeOnGet: false,
  updateAgeOnHas: false,
})

const replayKey = `${keyId}:${timestamp}`
if (seenTimestamps.has(replayKey)) {
  console.warn("Potential replay attack detected")
  throw errors.unauthorized("Authentication failed")
}
```

In the analyzed code, the application attempts to remember seen tickets using an in-memory storage called an LRU (Least Recently Used) Cache. However, there are two critical flaws in how this is implemented:

Missing State Update (The core bug): The code actively checks if a request's timestamp has been seen before using if (seenTimestamps.has(replayKey)). However, after the application successfully validates a new, unseen request, it never executes the command to save it (e.g., seenTimestamps.set()). Because nothing is ever written to the cache, the has() check will always return false. The application effectively has zero replay protection.

Missing Time-to-Live (TTL): The LRU cache is configured with a maximum size of 10,000 items (max: 10_000) but no expiration time (TTL). Even if the set() command were present, an attacker could generate 10,000 bogus requests to fill the cache. The cache would then automatically delete the

oldest entries to make room for new ones. If an attacker pushes a valid timestamp out of the cache while it is still within the 30-second validity window, they can successfully replay the original request.

Assets:

- apps/trading-api (single app only) [https://github.com/hknio/DreamX-Development___hodlbonds-api]

Status:

Fixed

Classification**Vulnerability Type(s):**

- Bypass Something

CVSS 4.0:

8.7 (AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N)

Weakness (CWE):

- [CWE-294 | Authentication Bypass by Capture-replay](#)

Severity:

High

Recommendations**Remediation:**

The authentication middleware must be updated to correctly store the signature/timestamp of validated requests. Additionally, the cache configuration must include a Time-To-Live (TTL) that strictly matches or exceeds the timestamp validity window (e.g., 30 or 60 seconds) so that entries are not evicted prematurely based on volume.

Update the LRU cache initialization to include a ttl property.

Ensure that `seenTimestamps.set(replayKey, true)` is called immediately after a request passes all HMAC and timestamp validation checks, but before calling `next()`.

F-2026-15404 - Race Condition in Order Processing Leads to Inconsistent State and Potential Data Loss - High

Description:

A race condition vulnerability exists in the order processing service (orderService). This type of issue occurs when a system's behavior depends on the sequence or timing of uncontrollable events, which in this case are separate network operations to a message queue and a database.

For a non-technical reader, this situation is analogous to sending a critical package via a courier and then recording the shipment in a logbook. The vulnerability is like the power going out in your office *after* the courier has already taken the package but before you could write down the tracking number in your logbook. The error-handling process, seeing no entry in the logbook, wrongly assumes the package was never sent and shreds the original order form. The result is that the package is in transit, but your internal system has no record of it ever existing, creating a serious discrepancy.

Evidence

File: DreamX-Development___hodlbonds-api-main/apps/trading-api/src/services/order.ts

Code:

```
const messageId = await adapter.executeSwap({ deadline, minAmountOut })
published = true
console.log({ messageId, clientId })

// Update reservation
const expiresAt = new Date(deadlineSeconds * 1000)
await db
  .update(tradeRequests)
  .set({
    messageId,
    expiresAt,
  })
  .where(eq(tradeRequests.id, reservationId))

return { clientId }
} catch (error) {
  if (!published) {
    await db.delete(tradeRequests).where(eq(tradeRequests.id, reservationId))
  }

  throw error
}
```

Technically, the orderService function first sends an order to be executed via a message publishing system (adapter.executeSwap) and then attempts to save the resulting message ID into its own database. The process is not atomic, creating two critical failure windows:

Failure After Publish, Before State Change: If the `adapter.executeSwap()` call successfully publishes the message but an exception occurs before the published flag is set to true, the catch block is executed. This block, believing the message was never sent, deletes the order reservation from the database (`db.delete()`). However, the message is already in the queue and will be processed, leading to an on-chain transaction for an order that no longer exists in the application's database.

Failure During Database Update: If the message is published successfully but the subsequent database update (`db.update()`) fails, the order record persists but without the critical messageId. This "orphan" record cannot be tracked or reconciled with the on-chain transaction, leaving it in an inconsistent state.

Assets:

- `apps/trading-api` (single app only) [https://github.com/hknio/DreamX-Development___hodlbonds-api]

Status:Fixed**Classification****Vulnerability Type(s):**

- Bypass Something

CVSS 4.0:

8.7 (AV:N/AC:H/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:H/SA:N)

Weakness (CWE):

- [CWE-362 | Concurrent Execution using Shared Resource with Improper Synchronization \('Race Condition'\)](#).

Severity:High**Recommendations****Remediation:**

The core issue is the lack of atomicity between publishing a message and updating the database. The system should be redesigned to handle failures gracefully without losing state.

In the catch block, replace `db.delete()` with an `db.update()` operation that sets the order's status to FAILED and stores the error details in a new column (e.g., `failureReason`).

This ensures that a record of every attempted order is preserved, which is crucial for auditing, debugging, and potential manual recovery.

F-2026-15401 - Plaintext Storage of Master Seed Phrase in Filesystem - High

Description:

It has been found that the master seed phrase for the application's cryptocurrency wallets persists as an unencrypted file on the server's filesystem, accessible by the executor application (`apps/executor/src/WalletManager.ts`).

To understand this issue, it is necessary to explain the concepts of a "Seed Phrase", an "HD Wallet", and "Plaintext Storage". In the cryptocurrency world, a "Seed Phrase" (or mnemonic phrase) is the ultimate master password. It is typically a sequence of 12 or 24 human-readable words that mathematically generates all the private keys for a user's digital wallets (this structure is known as an HD Wallet, or Hierarchical Deterministic Wallet). Anyone who knows this phrase has absolute, irreversible control over the funds inside. "Plaintext Storage" means this incredibly sensitive master password is saved in a normal computer file exactly as it is typed, without any encryption, masking, or secure vault protection.

While the project does use Google Cloud Secret Manager as the original source for the seed phrase — the deployment script (`init-deployment.sh`, lines 214–216) fetches it via `gcloud secrets versions access`, as documented in `DEPLOYMENT.md` (lines 387–389) — the fetched value is subsequently written to a file on disk and remains there for the lifetime of the container. This means the filesystem file acts as a persistent delivery mechanism, creating an at-rest exposure that negates some of the benefits of using a secrets manager in the first place.

The deployment does apply several compensating controls: the file is set to `chmod 600` permissions (`init-deployment.sh:134`), it is mounted read-only into the container (`docker-compose.main.yml:59`), and the container runs as a non-root user with UID 1001 (`Dockerfile:32-40`). Additionally, access to the underlying GCE VM is restricted by GCP IAM policies. Exploitation therefore requires privileged server-side access, such as SSH to the VM or a container escape, rather than being accessible to any arbitrary user or process.

In the application code, the `WalletManager` component's `getSeedPhrase()` method locates the file defined by the `SEED_PHRASE_FILE` configuration variable and reads its contents using `fs.readFileSync()`. This method is called during the application startup phase from `initWallets()` (line 121) and `buildMerkleTreeManager()` (line 167), both invoked once within the `start()` method (`WalletManager.ts:110–111`).

Evidence

File: DreamX-Development___hodlbonds-blockchain-executor-main/apps/executor/src/WalletManager.ts

Code:

```
/**
 * @return {string}
 */
getSeedPhrase(): string {
  const seedPhraseFile = baseConfig.SEED_PHRASE_FILE
  if (!fs.existsSync(seedPhraseFile)) {
    throw new Error(`Seed phrase file doesn't exist at ${seedPhraseFile}`)
  }

  return fs.readFileSync(seedPhraseFile).toString().replace(/\n/, "")
}
```

In the provided `WalletManager.ts` file, the `getSeedPhrase()` function retrieves the application's master cryptographic key by reading a file path specified in `baseConfig.SEED_PHRASE_FILE`. It checks whether the file exists on disk using `fs.existsSync()`, then reads the contents via `fs.readFileSync()`, converts it to a string, strips any trailing newline, and returns the seed phrase to the caller. Although the file is protected by filesystem permissions and container-level controls, the seed phrase nonetheless resides on disk in cleartext for the duration of the deployment. An attacker who gains privileged access to the host VM — for example, via a compromised SSH key, a container escape vulnerability, or misconfigured IAM roles — would be able to read the file and obtain full control over all derived wallets.

Assets:

- hodlbonds-blockchain-executor [https://github.com/hknio/DreamX-Development___hodlbonds-blockchain-executor]

Status:

Fixed

Classification

Vulnerability Type(s):

- Gain Information
- Gain Privilege

CVSS 4.0:

8.5 (AV:L/AC:L/AT:P/PR:H/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H)

Weakness (CWE):

- [CWE-312 | Cleartext Storage of Sensitive Information](#)

Severity:

High

Recommendations

Remediation:

Since Google Cloud Secret Manager is already the source of truth for the seed phrase (fetched via `gcloud secrets versions access` in `init-deployment.sh`), the recommended hardening is to eliminate the intermediate filesystem persistence by retrieving the secret directly at application runtime via the Secret Manager API.

1. Fetch via API at Runtime. Modify the application startup logic in `WalletManager.ts` to authenticate with Google Cloud and retrieve the seed phrase directly into memory using the `@google-cloud/secret-manager` SDK, replacing the current file-read approach in `getSeedPhrase()`. The application's existing GCP service account and IAM bindings can be reused for authentication.

2. Remove Filesystem Persistence. Update `init-deployment.sh` to remove the step that writes the seed phrase to a local file (line 134 and surrounding logic), and remove the corresponding read-only file mount from `docker-compose.main.yml` (line 59). This eliminates the on-disk exposure window entirely, removing the need for the current compensating controls (chmod 600, read-only mount).

3. Read Once and Clear. Since `getSeedPhrase()` is only called during the initialization phase — from `initWallets()` (line 121) and `buildMerkleTreeManager()` (line 167), both invoked once at startup within `start()` — fetch the seed phrase a single time, derive all necessary private keys into memory, and then overwrite and dereference the seed phrase variable so it does not persist in the application's memory space beyond initialization.

[F-2026-15399](#) - Authentication Bypass on Internal Endpoints in Non-Production Environments - Medium

Description:

It has been found that an Authentication Bypass exists in the `/internal/order-status` endpoint of the trading-api application (`apps/trading-api/src/routers/internal/order-status.ts`) when the application is running in a local development environment without OIDC configuration.

To understand this issue, it is important to clarify a few concepts. First, "Authentication" is the process of verifying a user's or system's identity, much like checking an ID badge at a secure facility. The system uses "OIDC" (OpenID Connect), which is an industry-standard digital ID badge often provided by cloud services like Google Cloud Pub/Sub. Second, software is usually deployed in different "Environments" (e.g., Development, Staging, and Production). "Production" is the live app used by real customers, while local development is a developer's own machine used for building and testing code.

Evidence

DreamX-Development__hodlbonds-api-main/apps/trading-api/src/routers/internal/order-status.ts

Code:

```
orderStatusRouter.post("/", async (c) => {
  if (env.SERVICE_URL && env.PUBSUB_SERVICE_ACCOUNT_EMAIL) {
    const authHeader = c.req.header("Authorization")
    const isValid = await verifyPubSubOidcToken(
      authHeader,
      env.SERVICE_URL,
      env.PUBSUB_SERVICE_ACCOUNT_EMAIL,
    )

    if (!isValid) {
      throw errors.unauthorized("Invalid OIDC token on order status update")
    }
  } else if (env.NODE_ENV === "production") {
    throw errors.internal(
      "SERVICE_URL and PUBSUB_SERVICE_ACCOUNT_EMAIL must be configured in production",
    )
  }
}
```

DreamX-Development__hodlbonds-api-main/apps/trading-api/src/index.ts

Code:

```
app.use(
  rateLimiter({
    windowMs: 60 * 1000, // Sustain @ 60/min
    limit: 60,
    standardHeaders: "draft-6",
    keyGenerator: rateLimitKey,
    skip: (c: { req: { path: string } }) => c.req.path.startsWith("/int
```

```
ernal/"),  
  }},  
  )
```

In the analyzed code, the application contains a flaw in how it handles its ID checks for internal services.

The code checks if the necessary configuration variables for OIDC are present. If they are not present, and the server's environment is set to anything other than production, the application skips the authentication check entirely and processes the request.

It should be noted that deployed environments (staging and production) are not affected by this bypass. The project's `Dockerfile` (line 36) sets `NODE_ENV=production` in all Docker images, and `cloudbuild.yaml` (lines 58, 63) injects both `SERVICE_URL` and `PUBSUB_SERVICE_ACCOUNT_EMAIL` for every deployment across all branches. Therefore, the bypass only materializes in local development when these environment variables are not configured.

Additionally, the rate-limiting configuration explicitly excludes any path starting with `/internal/` (`skip: (c) => c.req.path.startsWith("/internal/")`). This is an intentional design decision to accommodate Google Cloud Pub/Sub push delivery, and Cloud Run provides its own infrastructure-level request limits, so this does not present a Denial of Service risk.

Assets:

- apps/trading-api (single app only) [https://github.com/hknio/DreamX-Development___hodlbonds-api]

Status:

Fixed

Classification

Vulnerability Type(s):

- Bypass Something

CVSS 4.0:

6.9 (AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N)

Weakness (CWE):

- [CWE-288 | Authentication Bypass Using an Alternate Path or Channel](#)
- [CWE-306 | Missing Authentication for Critical Function](#)

Severity:

Medium

Recommendations

Remediation:

Security controls should be consistent across all environments. Avoid intentionally bypassing authentication logic based on the `NODE_ENV` variable, even in local development.

If using OIDC in local development is technically prohibitive, implement a secure fallback authentication mechanism (e.g., a static, strong `INTERNAL_API_KEY` passed via headers) that is exclusively used when `NODE_ENV !== 'production'`. This ensures that internal endpoints are never completely unauthenticated, even on a developer's machine.

[F-2026-15393](#) - Observable Timing Discrepancy in Webhook

Signature Validation - Low

Description:

It has been found a Timing Attack vulnerability in the Alchemy webhook signature validation process within the intake application (apps/intake/src/lib/signature-validation.ts).

To understand this issue, it is helpful to use the analogy of a burglar picking a combination lock. Imagine a lock that makes a faint "click" sound every time one of the numbers is aligned correctly. Even if the burglar doesn't know the full combination, they can slowly turn the dial, listen for the click, and figure out the code one number at a time. In the digital world, this is called a "Timing Attack." Instead of listening for clicks, an attacker measures exactly how many milliseconds the server takes to reject an incorrect password or digital signature.

Evidence

File: DreamX-Development__hodlbonds-api-main/apps/intake/src/lib/signature-validation.ts

Code:

```
function validateAlchemySignature(
  body: string,
  signature: string,
  signingKey: string,
): { matches: boolean; computed: string } {
  const computed = crypto.createHmac("sha256", signingKey).update(body, "utf8").digest("hex")
  const matches = signature === computed
  return { matches, computed }
}
```

In the analyzed code, the application verifies the authenticity of incoming webhooks by comparing a digital signature provided by Alchemy against a signature calculated by the server. However, it uses a standard strict equality operator (===):

```
const matches = signature === computed
```

Standard string comparison in programming languages like JavaScript is designed to be fast. It compares the strings character by character from left to right. As soon as it finds a character that does not match, it stops checking and immediately returns false (this is called "short-circuiting"). Therefore, comparing a signature where the first character is wrong takes slightly less time than comparing a signature where the first 10 characters are correct but the 11th is wrong.

Assets:

- apps/trading-api (single app only) [https://github.com/hknio/DreamX-Development___hodlbonds-api]

Status:**Fixed****Classification****Vulnerability Type(s):**

- Bypass Something

CVSS 4.0:

2.3 (AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N)

Weakness (CWE):

- [CWE-208 | Observable Timing Discrepancy](#)

Severity:**Low****Recommendations****Remediation:**

The application must perform a "constant-time" comparison when verifying cryptographic signatures, HMACs, or passwords. A constant-time comparison takes the exact same amount of time to execute regardless of how many characters match, completely eliminating the timing side-channel.

Replace the standard === operator with Node.js's built-in crypto.timingSafeEqual() function. Note that timingSafeEqual requires the inputs to be Buffer, TypedArray, or DataView objects of the exact same length.

[F-2026-15387](#) - Potential SQL Injection via Unparameterized sql.raw()

Usage - Info

Description:

An insecure coding pattern leading to a potential SQL Injection has been found in the ExecutionManager component of the hodlbonds-blockchain-executor-main project.

To understand this issue, it is important to briefly explain what SQL Injection (SQLi) is. SQL Injection is a severe security flaw that occurs when an application takes user-supplied data and inserts it directly into a database query without proper sanitization or formatting. This allows a malicious user to manipulate the structure of the database query, potentially tricking the database into deleting data, bypassing authentication, or revealing sensitive information. In secure development, this is prevented by using "parameterized queries," which strictly separate the SQL code from the data provided by the user.

In the analyzed code (apps/executor/src/ExecutionManager.ts), the application constructs SQL queries using the sql.raw() function. During this process, variables such as defaultRetryDeadline, regionRetryBlockDeadline, and executorRegion are inserted directly into the SQL string using template literals (e.g., '\$').

Currently, this code cannot be exploited to perform an actual SQL Injection attack. The input values are sourced from a configuration JSON file and are strictly validated by a Zod schema (AlphaNumericStringSchema) using the regular expression /^\\w+\$/ and specific Enums (EU1/US1). Because the \\w regex only allows alphanumeric characters and underscores, an attacker cannot input the special characters (like single quotes ' or semicolons ;) required to break out of the SQL string and inject malicious commands.

Evidence

File: DreamX-Development__hodlbonds-blockchain-executor-main/apps/executor/src/ExecutionManager.ts

Code:

```
const conditions = [
  eq(schema.queue.network, this.network),
  eq(schema.queue.status, status),
  sql`${schema.queue.earliest_try} <= now()`,
  sql`${schema.queue.next_attempt} is null OR ${schema.queue.next_attempt} <= now()`,
  // 1. If latest_try is null, check if we're inside the interval 2.
  // Check the actual date
  sql.raw(
    `((latest_try is null AND earliest_try + interval '${defaultRetryDeadline}' >= now()) OR latest_try >= now())`,
  ),
]

if (notIds.length > 0) {
```

```

conditions.push(not(inArray(schema.queue.id, notIds)))
}

if (status === Status.ERROR_RETRY) {
conditions.push(
sql.raw(
` (previous_attempt_region != '${executorRegion}' OR next_attempt +
interval '${regionRetryBlockDeadline}' < now())`,
),
)
}
}

```

Assets:

- hodlbonds-blockchain-executor [https://github.com/hknio/DreamX-Development___hodlbonds-blockchain-executor]

Status:

Fixed

Classification

Vulnerability Type(s):

- Execute Code
- Gain Information

Weakness (CWE):

- [CWE-89 | Improper Neutralization of Special Elements used in an SQL Command \('SQL Injection'\)](#).

Severity:

Info

Recommendations

Remediation:

Eliminate the use of `sql.raw()` for interpolating dynamic variables. Rely on the query builder's native parameterization capabilities (such as ``sql.fragment`` or equivalent, depending on the specific SQL library in use like `Slonik` or `Postgres.js`).

When dealing with PostgreSQL interval types, you cannot parameterize the string directly inside the interval `"` syntax. Instead, pass the value as a parameter and cast it to an interval type within the database (`::interval`).

[F-2026-15395](#) - Missing Blockchain Transaction Receipt Status

Validation - Info

Description:

It has been observed that the transaction confirmation logic in the executor application (`apps/executor/src/ExecutionResult.ts`) does not include an explicit check of the `receipt.status` field when evaluating blockchain transaction outcomes.

On blockchain networks like Ethereum, when a transaction is submitted, the network returns a "receipt" after processing. This receipt contains a `status` field (1 for success, 0 for failure/revert), which is the definitive indicator of whether the transaction's intended operation was actually executed. Reverted transactions still produce a receipt with a valid hash and gas data.

Upon analysis, the `processSingleTransaction()` function verifies only the existence of the receipt object and the presence of `receipt.hash` before calling `registerSuccess()`. There is no explicit check of `receipt.status` in the code. At first glance, this could appear to mean that reverted transactions would be incorrectly treated as successful.

However, the project uses ethers.js v6 (`"ethers": "^6.16.0"` per `apps/executor/package.json`), and this library's `TransactionResponse.wait()` method performs an internal status check. When a transaction reverts (`receipt.status === 0`), ethers.js v6 does not return the receipt — instead, it throws a `CALL_EXCEPTION` error. This behavior is confirmed in the ethers.js v6 source code, where the internal `checkReceipt()` function calls `assert(false, "transaction execution reverted", "CALL_EXCEPTION", ...)` when the status indicates failure.

In the current codebase, the `response.wait()` call (line 75–76) is wrapped in a `try/catch` block (lines 74–97). When a transaction reverts, the `CALL_EXCEPTION` thrown by ethers.js is caught at line 93, and `registerFailure()` is correctly invoked at line 94, marking the transaction as failed. Therefore, reverted transactions are already handled correctly through the library's exception mechanism.

Evidence

File: `DreamX-Development__hodlbonds-blockchain-executor-main/apps/executor/src/ExecutionResult.ts`

Code:

```
if (!receipt) {
  return this.registerFailure(
    `Task ${this.task.id} ${this.task.executor} response.wait() didn't
    get ${this.transactionConfirmations} confirmations and timed out af
    ter ${this.transactionTimeout / 1000} seconds`,
```

```

    )
  }

  this.registerGas(receipt.gasPrice, receipt.gasUsed)

  if (!receipt.hash) {
    return this.registerFailure(
      `Task ${this.task.id} ${this.task.executor} Receipt doesn't contain a hash`,
    )
  } catch (e) {
    return this.registerFailure(
      `Task ${this.task.id} ${this.task.executor} got exception while waiting for tx to be CONFIRMED: ${e}`,
    )
  }
  return this.registerSuccess([receipt.hash])
}

```

Assets:

- hodlbonds-blockchain-executor [https://github.com/hknio/DreamX-Development___hodlbonds-blockchain-executor]

Status:

Fixed

Classification

Vulnerability Type(s):

- Bypass Something

Weakness (CWE):

- [CWE-754 | Improper Check for Unusual or Exceptional Conditions](#)

Severity:

Info

Recommendations

Remediation:

As a defense-in-depth measure, consider adding an explicit `receipt.status` check before calling `registerSuccess()`. While ethers.js v6 currently handles reverted transactions by throwing a `CALL_EXCEPTION`, relying solely on implicit library behavior introduces a coupling risk: a future library upgrade, a migration to a different Web3 library (such as viem), or a change in ethers.js behavior could silently remove this safeguard. Adding an explicit check such as:

```

if (receipt.status === 0) {
  return this.registerFailure(
    `Task ${this.task.id} ${this.task.executor} transaction reverted (status=0)`,
  )
}

```

[F-2026-15402](#) - Use of Cryptographically Weak Hash Algorithm (SHA-1) - Info

Description:

The intake application (`apps/intake/src/utils/canonical.ts`) and the executor test suite (`apps/executor/test/ExecutorManager.test.ts`) utilize the SHA-1 hash algorithm to generate identifiers (`task_hash`). These hashes are used exclusively for task deduplication — specifically, at `Intake.ts:57` the generated hash is passed to an `onConflictDoNothing` database operation to prevent processing the same task twice. SHA-1 is not used here for any cryptographic security function (such as digital signatures, authentication, or integrity verification).

SHA-1 is an older hashing algorithm created in 1995 and considered deprecated by modern standards. In 2017, researchers demonstrated a practical collision attack (SHAttered), proving that two different inputs can produce the same SHA-1 hash. While this collision vulnerability is critical for cryptographic use cases, in the context of task deduplication the practical risk is significantly lower — an attacker would need to craft a collision that also satisfies the specific input format (`network|executor|earliestTry|canonicalData`), which is a highly constrained scenario.

Nevertheless, using a deprecated algorithm introduces unnecessary risk and contradicts security best practices. If the deduplication logic were ever extended or reused in a security-sensitive context, SHA-1 would become a direct vulnerability.

Evidence

File: DreamX-Development__hodlbonds-blockchain-executor-main/apps/executor/test/ExecutorManager.test.ts

Code:

```
    }: CreateDatabaseQueueItemParams): TaskQueueItem {
      const task_hash = crypto
        .createHash("sha1")
        .update(`${network}|${executor}|${earliest_try_unix}|${JSON.stringify(data)}`)
        .digest("hex")

      return {
```

File: DreamX-Development__hodlbonds-blockchain-executor-main/apps/intake/src/utils/canonical.ts

Code:

```
export function computeHash(message: Message): string {
  const canonicalData = canonicalize(message.data)
  const sha1 = crypto.createHash("sha1")
  sha1.update(`${message.network}|${message.executor}|${message.earli
```

```
estTry}}|${canonicalData}`)  
return sha1.digest("hex")  
}
```

The application concatenates task properties (network, executor, earliestTry, canonicalData) into a single string, passes it through `crypto.createHash("sha1")`, and uses the resulting 40-character hex string as a deduplication key via `onConflictDoNothing`.

Assets:

- hodlbonds-blockchain-executor [https://github.com/hknio/DreamX-Development__hodlbonds-blockchain-executor]

Status:Fixed**Classification****Vulnerability Type(s):**

- Denial of Service
- Gain Information

Weakness (CWE):

- [CWE-328 | Reversible One-Way Hash](#)

Severity:Info**Recommendations****Remediation:**

Replace the deprecated SHA-1 algorithm with a modern hashing algorithm such as SHA-256 (SHA-2 family). Even though the current usage is limited to deduplication and does not serve a direct cryptographic security function, migrating to SHA-256 eliminates the use of a known-weak primitive, aligns with current industry best practices, and prevents potential risk if the hashing logic is reused in a security-sensitive context in the future. The change is straightforward — replacing `"sha1"` with `"sha256"` in `crypto.createHash()` — and has no meaningful performance impact.

Disclaimers

Hacken Disclaimer

The application given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the application's source code, its deployment, and functionality (performing the intended functions) being the focus of our analysis.

The report contains no statements or warranties regarding the identification of all vulnerabilities or the absolute security of the code. The report covers only the code that was submitted and reviewed, and therefore may not remain relevant after any modifications have been made. This report should not be considered a definitive or exhaustive assessment of the utility, safety, or bug-free status of the application, nor should it be taken as a guarantee of the absence of other potential issues.

While we have exerted our best efforts in conducting the analysis and producing this report, it is crucial to understand that this report should not be the sole source of reliance for ensuring the security of the application. We strongly recommend undertaking multiple independent audits and establishing a public bug bounty program to enhance the security posture of the application.

English is the original language of this report. The Consultant is not liable for any errors or omissions in any translations of this report.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Applications, whether decentralized apps (DApps) or other types of applications, are deployed and run within specific environments that may include various platforms, programming languages, and other related software components. These environments and components can have inherent vulnerabilities that might lead to security breaches. Consequently, the Consultant cannot guarantee the absolute security of the audited application.

Appendix 1. Severity Definitions

Findings are categorized based on their potential impact and assigned a severity level using the [Common Vulnerability Scoring System \(CVSS\) version 4.0](#):

Severity	Description
Critical	These issues present a major security vulnerability that poses a severe risk to the system. They require immediate attention and must be resolved to prevent a potential security breach or other significant harm.
High	These issues present a significant risk to the system, but may not require immediate attention. They should be addressed in a timely manner to reduce the risk of the potential security breach.
Medium	These issues present a moderate risk to the system and cannot have a great impact on its function. They should be addressed in a reasonable time frame, but may not require immediate attention.
Low	These issues present no risk to the system and typically relate to the code quality problems or general recommendations. They do not require immediate attention and should be viewed as a minor recommendation.

Appendix 2. Scope

The scope of the project includes the following:

Scope Details	
Repository	https://github.com/DreamX-Development/hodlbonds-blockchain-executor
Commit	35c049a (main branch)
Repository	https://github.com/DreamX-Development/hodlbonds-api
Commit	e8b188c (main branch)

Appendix 3. Additional Valuables

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), the [Penetration Testing Execution Standard \(PTES\)](#), and the [OWASP Testing Guide](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.